Session 7: Conversational Interfaces with LLMs

Objective:

By the end of this session, students will be able to:

- Understand the limitations of traditional prompt-based generation in multi-turn conversations, including a lack of memory, role-awareness, and behavioral control.
- Explain the structure and components of Chat Completion APIs, including the roles of **system**, **user**, and **assistant** messages in enabling contextual and coherent dialogue.
- Demonstrate how to use the **system** message to guide assistant behavior across a session and maintain consistent tone, style, or domain expertise.
- Describe how Chat Completion works under the hood, and explain why LLMs do not have real memory but rely on prompt engineering and structured message history.
- Apply effective context management strategies—such as summarization, truncation, and prioritization—to maintain conversation relevance within the limits of the model's context window.

In the previous sessions, our interaction with language models followed a simple prompt-response paradigm. The workflow was straightforward:

1. We would write a prompt — e.g.,
   "Summarize the following paragraph..."

2. Send this prompt to the model.

3. The model would return a one-time response based solely on the input prompt.

4. If we needed to continue the interaction or maintain continuity, we would manually append previous outputs to the next prompt and resend it.

While effective for one-off tasks, this approach becomes increasingly cumbersome when we attempt to build interactive systems, like chatbots, virtual assistants, or helpdesk agents.

Why Text Generation Falls Short in Conversations

While prompt-based generation is powerful for many single-turn use cases, it becomes significantly limited when applied to **interactive, multi-turn scenarios** — such as chatbots, virtual assistants, or customer service agents.

Let's break this down with real-world analogies and examples to better understand the limitations.

Imagine our simple chatbot is designed to answer questions about movies.

The Simple Movie Chatbot (Prompt-Based)

User: "Who directed Inception?"

Chatbot: "Christopher Nolan."

Now let's see where it falls short.

While our simple movie chatbot works for a single question, it quickly becomes limited when we try to have a real conversation.

1. No Built-in Memory

Let's continue our conversation with the simple movie chatbot:

User: "Who directed Inception?"
Chatbot: "Christopher Nolan."
User: "What other movies did he direct?"
Chatbot: "I'm sorry, I don't have enough information to answer that. Could you please provide the name of the director again?"

Problem: Our simple chatbot has no memory. When the user asks "What other movies did *he* direct?", the chatbot doesn't remember that "he" refers to Christopher Nolan from the previous turn. Each time you send a prompt, it's like starting a brand new conversation from scratch.

It's like talking to a customer support agent who asks for your order ID every single time you speak, even if you just gave it to them in the previous sentence.

2. Manual Context Stitching

To make our simple chatbot "remember," we'd have to manually combine the previous turns into a single, longer prompt for each new query.

User: "Who directed Inception?"
Chatbot: "Christopher Nolan."

Now, for the next question, we'd have to send this as the prompt:
Prompt to Chatbot: "User: Who directed Inception? Chatbot: Christopher Nolan. User: What other movies did he direct?"

Chatbot's (hypothetical, if it could handle this long prompt) response: "Christopher Nolan also directed The Dark Knight, Interstellar, and Dunkirk."

Problem: This is "Manual Context Stitching." We, the developers (or the user, in a very basic system), have to manually paste all previous messages into every new prompt.

Developer friction:

- **Increases prompt length:** The more turns, the longer the prompt, which can slow down processing and potentially cost more per API call (if based on token usage).
- **Risk of repetition or redundancy:** It's easy to introduce errors or irrelevant information if you're constantly copying and pasting.
- **Manual effort:** It's a significant burden to write code that manages and formats this growing message history. Imagine trying to keep a 50-turn conversation coherent this way!

It's like trying to write a play where, before each new line is spoken, you have to read out the *entire script from the beginning* to remind the audience of the plot. It's unmanageable.

3. Lack of Role Awareness

In our simple prompt-based chatbot, there's no inherent way for the model to distinguish between the user's input, the chatbot's previous response, or any hidden instructions. It just sees a continuous block of text.

Consider this prompt we might send to our chatbot:
Prompt: "The user asked 'Who directed Inception?'. The assistant responded 'Christopher Nolan.'. Now the user asks 'What about Interstellar?'"

Problem: The model just sees a string of text. It doesn't natively understand that "The user asked..." is a user's query, and "The assistant responded..." is its own previous output. It can't differentiate between these "voices" or roles.

In an intelligent assistant, distinguishing between roles is critical:

- What's an **instruction** (e.g., "Act like a helpful movie expert")?
- What's a **user query** ("Tell me about the plot of Arrival")?
- What's a **model response** ("Arrival is a science fiction film...")?

Prompt-only systems offer no native structure to support this, making it hard to build robust conversational flows.

4. No Native Behavior Control

Let's say we want our simple movie chatbot to always respond in a very concise way, or perhaps always recommend family-friendly movies.

With our simple prompt-based chatbot, we'd have to include that instruction in *every single prompt*:

Prompt 1: "Respond concisely. Who directed Inception?"
Chatbot: "Christopher Nolan."

Prompt 2: "Respond concisely. What other movies did he direct?"
Chatbot: "The Dark Knight, Interstellar, Dunkirk."

Problem: We can't consistently instruct the model to maintain a certain behavior across turns unless we repeat those instructions every single time. If we forget, the chatbot might revert to a default behavior.

5. Not Scalable for Real Applications

Let's imagine we want to build an enterprise-level movie recommendation service based on our simple chatbot.

Problem: The more turns in the conversation, the more context you have to manually manage. This makes the architecture brittle and hard to maintain.

Real-world analogy: Building an enterprise chatbot using this prompt-based interaction is like running a call center where each agent starts every call with zero history of the customer. Every time a customer calls back, a new agent picks up, and the customer has to repeat their entire problem from scratch.

You lose efficiency, personalization, and scalability — all of which are essential for production-level conversational systems. It becomes impossible to deliver a smooth, continuous, and personalized experience.

This is where **Chat Completion** comes in — it mimics real human conversation by allowing structured, multi-turn dialogues between you and the model.

Let's explore how that works.

1. Chat Completion API:

In the real world, conversations aren't just a series of disconnected questions and answers. They're dynamic, context-rich, and role-driven.

When you speak to a teacher, a customer service agent, or even a friend, you expect them to:

- Remember what you said earlier
- Respond in a tone that fits the relationship
- Adapt based on the flow of the conversation

Yet when we interact with language models using plain prompts, we lose all of that.
There's **no memory**, **no sense of role**, and **no behavioral consistency**.

Chat Completion APIs are designed to make model interactions **feel like real conversations** — not just disconnected generations.

It refers to the logic that takes a structured chat history (with roles like user/assistant/system) and uses a language model (e.g., Gemini-2.5-flash) to generate the next assistant message — the completion.

So, it has two main responsibilities:

Formatting the chat history into a flat input for the model.

Generating the assistant's response — the actual language output.

Raw LLMs (like GPT, Gemini, Claude) are trained to **predict the next token** — they don't naturally understand "chat".
Chat Completion adds structure, turning a text-generation model into a **chatbot or assistant.**

It makes conversations possible

It is a way of interacting with a language model using **structured, role-based messages** — instead of one long prompt.

It allows you to create **context-aware conversations** by passing a list of message objects, each with a **role** and **content**. You send a **structured list of messages**, each tagged with its **role**:

- **system** for setting the assistant's personality
- **user** for the human inputs
- **assistant** for model responses

This structured approach gives your application the power to:

- Maintain context across multiple turns
- Clearly separate who said what
- Control the assistant's tone, behavior, and style — persistently

Now that we've been introduced to Chat Completion, it's important to pause and realign how we think about interacting with language models.

Chat-based interaction isn't just a new format — it's a shift in **how we frame communication** with an AI system.

Unlike traditional prompting, where the model sees just a single block of text, chat completion introduces **structure**, **sequence**, and **intention** to the interaction. It mirrors the way humans engage in conversation: with turns, memory, and context.

Think About a Real Conversation:

When two people talk, several things happen naturally:

- There's a clear back-and-forth — a sense of turn-taking
- Each person **remembers** what was said before
- The roles are understood — one person asks, the other responds
- The tone remains **consistent** — friendly, formal, casual, etc.

This isn't just social etiquette — it's what makes communication *coherent and meaningful*.

What makes the Chat Completion approach powerful is that it brings this **human-like structure** into our interaction with a model. Each message in the conversation is no longer just a string of text — it has **a purpose** and **a perspective**.

It tells the model:

- *Who is speaking* (user, assistant, or system)
- *What the message represents* (a question, a reply, or an instruction)
- *Where it fits* in the conversation history

But if we really want to **build** or **control** such interactions in practice — whether in a chatbot, tutor, or assistant — we need to understand **how these conversations are actually structured under the hood.**

So, **what does a chat message really look like?**

Surprisingly, it's not complicated at all.

Every Message Has Just Two Essential Ingredients:

a. Role
b. Content

Every message — whether it's the user speaking, the assistant replying, or a system instruction guiding behavior — is built from just **two simple keys**

In a real conversation, you always know who is speaking.
Whether it's your teacher, a teammate, a customer service agent, or even yourself, the meaning of a message is shaped not just by *what* is said, but *who* is saying it.

This is exactly what the **role** field captures in Chat Completion.

> The **role** defines the speaker's identity and intent in the conversation flow.
> It helps the model determine *how to interpret* a message and *how to respond appropriately*.

Let's imagine a simple conversation between a student, a tutor, and a moderator:

- The student asks: "Can you explain photosynthesis?
- The tutor replies: "Sure! Let me break it down."
- The moderator (off-stage) says: "Please be concise and use simple language."

Each of these people has a distinct role:

- One is asking
- One is answering
- One is setting expectations

That's exactly how LLM chat works — using roles to make sense of who's talking and why.

1.1 In Chat Completion, There Are Only 3 Roles:

1. **system** — The Invisible Guide

> Think of it as the *director* of the conversation.

This role defines how the assistant should behave throughout the chat:

- What kind of personality it should have (friendly, sarcastic, expert)
- What tone or language style it should follow (formal, concise, creative)
- Any fixed boundaries or constraints (e.g., don't answer questions outside math)

Used only once at the start of the conversation.

For Example:

In a customer support center, the agent may get a note before a call:
"Speak politely. Don't give out personal opinions. Resolve queries within 2 minutes."

That's the **system** message — it doesn't participate in the chat, but it shapes how the assistant responds.

2. **user** — The Human Speaker

> This is the person driving the conversation: asking questions, giving commands, or sharing input.

The **user** role represents the real user — a customer, learner, developer, or whoever is interacting with the assistant.

It could be:

- A student asking a question
- A customer making a request
- A developer giving an instruction

The assistant sees every user message as something it needs to respond to.

For example, You're typing into a chatbot interface or talking to a smart assistant like Google Assistant.

3. **assistant** — The LLM's Voice

> This is the model's side of the conversation.

Whenever the model generates a response, it is represented as an **assistant** message.

You don't write this role yourself, unless you are simulating a previous reply during a long conversation history.

For Example:

This is the support agent replying back, the tutor giving an answer, or the AI responding on-screen.

It reflects how the LLM interprets all previous messages (from system + user) and formulates an intelligent, role-aligned reply.

Now that we understand the **role** field — which defines **who** is speaking in a Chat Completion interaction — let's focus on the second essential building block:

The content tells us exactly *what* is being said.

Whether it's a prompt from the user, a behavioral guideline from the system, or a generated response from the assistant, the actual text is carried inside the content field

So, What Exactly Is content?

The **content** field is simply a string of text that holds the message. It's the actual body of the conversation — what the speaker (as defined by the role) is saying at that point.

It can be:

- A question: "*What is machine learning?*"
- A command: "*Write a Python function to check for prime numbers.*"
- A reply: "*Sure! Here's the code for that*."
- A behavioral setup: *"You are a helpful, concise assistant who replies in bullet points.*"

For example:

```
[

System: You are a helpful travel planner.

User: What's a good 3-day itinerary for Goa?

Assistant: Sure! Here's a suggested 3-day plan for Goa:

  Day 1: Arrive in North Goa. Explore Calangute and Baga beaches. Try water sports. Evening at a beach shack or Tito's Lane.

Day 2: Visit Aguada Fort in the morning, then head to Anjuna for lunch and flea market shopping. Sunset at Chapora Fort.
```

Day 3: Relax at Morjim or Ashwem beach. Enjoy a seafood lunch. Optional visit to Panjim for heritage walk and cruise on the Mandovi River before departure.

User: Can you also suggest a couple-friendly resort near Morjim?

]

So far, we've learned that every message in a Chat Completion setup is defined by just two things:

- **role** — who is speaking
- **content** — what they're saying

We now understand that:

- The **user** role represents the prompts or questions we send to the model
- The **assistant** role represents the responses generated by the model itself

This back-and-forth forms the **core interaction loop** in any chat-based system.

But here's the real question:

> If this back-and-forth works fine with just user and assistant…
> Then **why do we even need a system role?**

Why not just send a user question and get an assistant answer?

To truly understand this, let's go **one level deeper**.

Let's explore the **purpose and power of the system role** — a role that doesn't directly "talk" during the conversation but silently **controls everything** about how the assistant behaves.

Think of it like this:

You're not just chatting with an LLM —
You're **designing how it should think, speak, respond, and behave**.

And that's where the **system** prompt comes in.

1.2 The system Role in Chat Completion:

Imagine you just hired a new customer support executive at a fintech startup. Before they ever respond to a user ticket, you give them a clear set of instructions:

> "You are a professional, empathetic support agent. You always have to greet the user by name, respond in under 3 sentences, and never make assumptions. If the issue relates to billing, escalate it to Level 2. Understand?"

They don't repeat this script in every reply — but every message they send is **shaped by it**.

That's what the **system** prompt is:

A **system** message is used to set the **behavior**, **tone**, **domain expertise**, or **output structure** of the language model **before** the user starts asking questions.

It influences how the assistant interprets all subsequent messages, but **it doesn't get repeated or spoken by the model itself**.

1.3.1 Why the System Prompt Matters?

The **system** prompt is the **invisible instruction manual** that tells the assistant **who it should be**, **how it should behave**, and **how it should respond** — even before the user says a word.

Now let's break this down in depth:

1. It Sets the Initial *Mindset* of the Model:

When the Chat Completion API is called, the model doesn't "start thinking" at the user message — it begins **processing from the very first token of the input**. And that first token?
It's usually the **system** message.

> The system prompt becomes the **foundation** for all reasoning, tone, and formatting choices the model makes across the entire conversation.

It's like **priming a machine before it starts** — once it's been told to act like a formal legal assistant or a coding tutor, every response it generates follows that initial lens.

2. All Responses Are Filtered Through This Instruction:

Once the system prompt has defined the assistant's **persona**, every assistant reply is influenced by it — whether directly or subtly.

Example:

```
{ "role": "system", "content": "You are a poetic assistant. Always answer in rhymes." }
```

Later, if the user asks:

"What is the moon made of?"

The assistant might respond:

"The moon's not cheese, though tales are bold,
It's dust and rock, both grey and cold."

Without the system prompt, this same model may have responded with a plain factual answer.

The system message acts as a **behavioral filter** that wraps every assistant message in the tone or constraints you've defined.

3. Without It, Model Behavior Becomes Generic or Inconsistent:

If no system prompt is supplied, the model behaves with **default settings**:

- Tone: neutral or formal
- Output: mixed formatting (paragraphs, sometimes unordered)
- Behavior: generic assistant (helpful, but not specialized)

This works fine for casual Q&A, but it fails when:

- You want **domain-specific expertise** (law, medicine, finance, etc.)
- You want a **specific tone or brand voice** (funny, concise, formal)
- You require **structured outputs** (JSON, Markdown, tabular)

Example:

Without system prompt:

User: "Summarize this in 3 bullet points."
Assistant: Returns a paragraph, or 5+ bullets, or includes commentary.

With system prompt:

"*You are a structured assistant. Always summarize responses in exactly 3 bullet points. Never add commentary.*"

Now the assistant aligns to your expectations — **every time**.

Some Sytem Instruction Examples:

1. You are a friendly and informal assistant. Use emojis, ask follow-up questions, and keep your tone light and cheerful."

2. You are a knowledgeable high school physics teacher. Explain concepts clearly with examples,

and avoid technical jargon unless necessary.

3. You are a poetic assistant. Always answer in rhymes or verses, and maintain a lyrical tone in your responses.

4. You are a helpful and polite assistant. Answer questions accurately and concisely.

We've just seen how Chat Completion helps structure conversations — with clear roles, turn-by-turn exchanges, and consistent assistant behavior.

On the surface, it really feels like the model is *remembering* everything.

Ask it a follow-up question, and it answers correctly.
Use a certain tone, and it maintains that tone across responses.
Mention something once, and it seems to carry that forward intelligently.

So it's natural to wonder:

I*s the model actually storing previous messages somewhere?*
Does it have some kind of memory like a human?
Is it learning and evolving during the conversation?

Because this feels quite different from what we saw earlier with plain text generation, where:

- Every new prompt was standalone
- No previous context was remembered
- And the assistant had no continuity across responses

So what makes Chat Completion feel so coherent — as if it's thinking in a continuous thread?

What's really going on behind the scenes?

To answer that, we need to take a look *under the hood* and understand how Chat Completion actually works.
How the model processes message history, how it uses context, and what we really mean when we say "it remembers."

Let's explore that

2. How Does Chat Completion Actually Work?

Before we explore how *chat completion* works in LLMs, it's important to take a step back and understand a fundamental concept:

How does an LLM actually "chat"?

In other words — *how does a large language model generate responses to user queries in a conversational setting?*

This foundation is essential because chat completion is not a magic switch. It's built on the core mechanism of how LLMs **predict and generate text**, token by token, based on previous context. Once we understand that, it becomes much easier to see how structured conversations are built on top of it

At its core, a Large Language Model (LLM) doesn't generate an entire response all at once. Instead, it works **step by step**, predicting the next most likely word (or token) based on everything that has been seen during its training

2.1 But What Does an LLM *Really* Do?

An LLM (Large Language Model) **writes text by guessing one piece at a time** — kind of like a supercharged **autocomplete** that keeps going… word by word… or piece by piece.

It doesn't know what it's going to say in advance — it makes each new word (or part of a word) by **predicting what should come next**, based on what it has already seen.

Step-by-Step: How It Predicts the Next Token

1. Break the sentence into small pieces (tokens)

Before the model can work with a sentence, it splits it into **tokens**, which can be words, parts of words, or even punctuation.

Example:
"The sky is blue" → becomes → ***["The", " sky", " is", " blue"]***

These pieces are easier for the model to process.

2. Convert tokens into numbers

The model doesn't understand text directly — it thinks in **numbers**.

Each token gets turned into a **vector** (a list of numbers). These numbers capture some meaning — like how the word relates to others.

Think of it like giving each word a "digital fingerprint" that the model can understand.

3. Understand the context using attention

This is the **Transformer magic**. The model looks at **all previous words** to understand what's going on.

It uses something called **attention** to decide:

- What words are important
- How do they relate to each other

Example:
In the sentence:
"The cat sat on the ___"
The model figures out:

- "cat" is the subject

- "on the" means we expect a location
  → So likely next words are **"mat"**, **"sofa"**, or **"floor"** — **not** "banana" or "rocket".

4. Guess the next token using probabilities

Now, the model makes a list of possible next tokens and gives a **score (probability)** to each.

Example:

```
"mat" → 0.65

"sofa" → 0.20

"floor" → 0.10

"banana" → 0.01

"elephant" → 0.0001
```

It's saying:
"I'm 65% sure it's 'mat', but there's a small chance it could be something else."

5. Pick the next token

Once a language model computes the probability distribution over all possible next tokens, it needs to decide **which token to actually generate**. This is where **decoding strategies** come into play — techniques used to **sample** or **select** the next token based on the predicted probabilities.

Here are the most commonly used decoding methods, each with distinct behavioral trade-offs:

5.1 Greedy Decoding

Definition: Always selects the token with the **highest probability** at each step.

- **Example:**
  Given probabilities:
  `mat → 0.65, sofa → 0.20, floor → 0.10, ...`

  → The model always chooses **"mat"**.
- **Pros:**
  - Deterministic (same output every time)
  - Fast and simple
  - Suitable for factual, structured tasks
- **Cons:**
  - Lacks creativity
  - Can get stuck in repetitive or dull output
  - Doesn't explore alternate valid continuations

5.2 Random Sampling

Definition: Picks the next token **randomly**, weighted by their probability scores.

- **Example:**
  With the same distribution, **"mat"** is most likely, but **"sofa"** or **"floor"** could still be chosen.
- **Pros:**
  - Enables creative and varied output
  - Can produce surprising, non-repetitive results
- **Cons:**
  - May generate incoherent or inconsistent responses
  - Not ideal for high-precision or domain-specific applications

5.3 Top-k Sampling

Definition: Limits token selection to the **top k tokens** with the highest probabilities. The final token is then **randomly sampled** from this subset.

- **Example:**
  If k=3, the model only samples from `["mat", "sofa", "floor"]`.
- **Pros:**
  - Balances between precision and creativity
  - Reduces the chance of choosing extremely low-probability, nonsensical tokens
- **Cons:**
  - Choice of k is arbitrary and may not adapt well to all situations
  - Can still ignore rare, but contextually relevant tokens

5.4 Top-p Sampling (Nucleus Sampling)

Definition: Instead of a fixed number of top tokens (like Top-k), Top-p selects the **smallest set of tokens whose cumulative probability exceeds p** (e.g., 0.9). Sampling is then done from this dynamic subset.

- **Example:**
  The model may include only `["mat", "sofa", "floor"]` if they collectively account for 90% of the probability mass.

- **Pros:**
  - Adaptive to the distribution — chooses fewer tokens when confident, more when uncertain
  - Encourages more natural, human-like variability
- **Cons:**
  - Slightly more complex to configure
  - Still probabilistic — results vary each time

6. Do it again. And again. And again…

Once it picks a token, it adds that to the sentence and repeats the whole process — predicting the next token, and the next, until:

- It reaches the end
- Or the user stops it

That's how you get full answers, stories, poems, code, or essays — **one token at a time**.

And that's how a model predicts the next words and then completes the response

So let's understand how LLM actually does chat. We can take an example of an actual chat completion system, like ChatGPT or Gemini

Let's make this real.

Step 1: Try this in ChatGPT
Open ChatGPT and type:

> *"My name is Ninja"*

Then hit enter.

You'll see the model respond with something like:

> *"Nice to meet you, Ninja! How can I help you today?"*

Now pause here for a second and ask:

2.2 What just happened technically after you hit enter?

Let's walk through it, step-by-step, behind the scenes.

1. Your Message is Wrapped as a Role-Based Object:

When you typed:

"My name is Ninja."

It is wrapped by the system as a structured message like:

{ "role": "user", "content": "My name is Ninja" }

Behind the scenes, ChatGPT maintains a full list of messages in memory like:

```
[
  { "role": "system", "content": "You are a helpful assistant." },
  { "role": "user", "content": "My name is Ninja" }
]
```

2. The Entire Message List is Flattened into Text:

Before sending it to the model, ChatGPT flattens this structured list into a single prompt string, something like:

System: You are a helpful assistant.
User: My name is Ninja

This entire string is the actual input prompt.

3. The Text is Tokenized:

Now this flattened string goes through a **tokenizer**, which breaks it into small units that the model understands.

Example:

- "My" → token 3798
- "name" → token 1715
- "is" → token 318
- "Ninja" → token 92005

This list of tokens is what gets passed to the LLM.

4. LLM Starts Predicting — Token by Token:

Now the model does what it was trained to do:

- It looks at all the input tokens,

- It uses its internal neural network to **predict the next most likely token,** one at a time,
- And generates the reply — step-by-step — like:

  "Nice" → "Nice to" → "Nice to meet" → "Nice to meet you," → …

Each of these words is actually predicted **token-by-token**, in a streaming generation loop.

5. The Tokens Are Decoded into Text:

Once generation is complete, the token list is decoded back into readable text:

"Nice to meet you, Ninja! How can I help you today?"

6. This is Sent Back and Displayed:

The assistant's message is now wrapped again as:

{ "role": "assistant", "content": "Nice to meet you..." }

It is added to the conversation history — so the model will consider it for future turns.

Let's continue the chat, and now in the **next message**, you type:

"What is my name?"

And ChatGPT replies:

"*Your name is Ninja."*

How????? How did it remember that?

It **feels** like ChatGPT has remembered your name.
But here comes the **big question** we need to ask:

"Does the model actually remember everything I've said so far?"
"Does it store every message I've typed somewhere?"
"Is there a memory built into the model?

Let's begin with this fundamental question that often comes up when people first interact with Chat Completion:

Does a large language model (LLM) store memory during a conversation?

At first glance, it certainly feels like it does.
When you ask a follow-up question or refer to something mentioned earlier, the assistant responds as if it remembers what you said.

But here's the truth:

> **The LLM does not have any memory of its own.**
> It doesn't store past chats, learn across turns, or build a long-term memory.

So, what's really happening?

3. It's Not Memory — It's Prompt Engineering Behind the Scenes

The illusion of memory is created by **how we send data to the model** — not by anything the model stores internally.

Let's break it down:

3.1 In Text Completion:

You send a single prompt (e.g., a question or a paragraph), and the model responds based only on that one input.
There's **no built-in mechanism** to remember previous prompts.
If you want continuity, you have to manually stitch previous context into the new prompt.

Example 1: Sentence Completion

Prompt:
The tallest mountain in the world is

Model Response:
Mount Everest.

Here, the model simply **predicts the next words** that make sense.

Let's say you now want to ask a follow-up question:

How tall is it?

You send that alone, as a separate new prompt.

But this time, the model may respond:

I'm not sure what "it" refers to. Could you clarify?

Why?

Because in **Text Completion**, there's no memory.
The model never saw your previous sentence —

> "The tallest mountain in the world is Mount Everest."

So it can't understand what "it" refers to.

To solve this limitation, we use **Chat Completion** — a structured way of talking to the model that sends the **entire conversation history** as a list of messages.

3.2 In Chat Completion:

It might feel smarter, but underneath the hood, it's still **just prompt-based generation**.

The only difference is:

> You're sending a full list of messages — the entire chat history — as a **single prompt** in structured format.

This history includes:

- The **system** instruction
- Every **user** input so far
- Every **assistant** reply so far

All of that is bundled together and sent to the model as one large context window.

What really happens when you send a Chat Completion request to an LLM like Gemini?

Let's break down the entire journey — from writing your code to receiving a response, because at the end, the model doesn't **remember** — it simply **reads everything we send it**, processes that context, and generates the next response accordingly.

1. Your Code (Where It All Begins)

You initiate a request using your code — often through the Gemini SDK or HTTP API.

model.generate_content(messages)

Here, you define:

- A **messages** list representing the full conversation
- Model configuration
- Optional parameters: **temperature, top_p, max_tokens,** etc.

At this point, the request is ready to be formatted and prepared for inference.

2. Structured Messages (Role-Based Conversation Format)

Your input is structured using three message roles:

Role	Purpose
system	Sets behavior, tone, constraints, domain expertise
user	Actual user inputs — questions, instructions, etc.
assistant	Previous replies from the model, if any

Example:

```
[
  {"role": "system", "content": "You are a friendly and concise assistant."},
  {"role": "user", "content": "What's the capital of Japan?"},
  {"role": "assistant", "content": "Tokyo."},
  {"role": "user", "content": "Give me three facts about it."}
]
```

This role-based structure enables conversational flow and personality control.

3. Prompt Formatter / SDK Serialization

The SDK or backend now transforms the structured messages into a **single, flattened text string** suitable for the model to understand.

Example (Internal Format)

System: You are a friendly and concise assistant.

User: What's the capital of Japan?

Assistant: Tokyo.

User: Give me three facts about it.

> The model will see the entire prompt as one context block, with turn-by-turn structure, preserved history, and behavioral framing.

Why Is This Necessary?

Because LLMs like GPT and Gemini are stateless. They do not store memory between requests.

4. Final Prompt String (Ready for Inference)

At this point, the complete context (system behavior + all user and assistant turns) is assembled into a single string, which becomes the final **input** to the model.

This context is **not remembered automatically**. Every time you send a new message, you must **re-send the full message history**.

5. API Request → Gemini / LLM Endpoint

Your application makes a real-time HTTPS API call to the LLM endpoint.

Payload includes:

- The final prompt string
- Model parameters (e.g., max_tokens, temperature)
- Authentication keys or tokens

This request is now handed off to the LLM infrastructure.

6. LLM Processes Context & Generates Response

Once your request reaches the model server:

> The LLM processes the **entire context you provided** — from the system message to every user-assistant interaction — and generates the next response.

- It doesn't store previous chats or build persistent memory
- It simply uses the provided message history (within the model's context window)
- It generates the response one token at a time, predicting based on all previous inputs

You can think of this as a black box:

- You send a structured prompt
- The model returns the next reply

7. Generated Tokens → Model Output Text

The LLM produces a sequence of **tokens** (internal representations of words or subwords).

Depending on how you've configured your request:

- You may receive these tokens **incrementally** (streamed)
- Or you receive them **all at once** (as a full response)

These tokens are then **decoded into a human-readable string**.

"Sure! Here are three facts about Tokyo:
1. It's the capital of Japan.
2. It's the most populous city in the world.
3. It hosted the 2020 Summer Olympics."

8. SDK Wraps Response with role=assistant

Once the model's output is ready, the SDK wraps it in a structured response format:

```
{

  "role": "assistant",

  "content": "Sure! Here are three facts about Tokyo..."

}
```

This makes the assistant's message compatible with your existing conversation format — so it can be appended and reused in the next interaction.

9. Final Output Returned to You

The final structured response is returned to your application, either as:

- A **Python object** (via SDK)
- Or **raw JSON** (via HTTP response)

From here, you can:

- Render the output in your chat interface
- Store it as part of the chat history
- Feed it back into the message list for future turns

4. Hands-on Session:

4.1: Recap What We've Learned So Far

Let's start by revisiting the basic approach, where we **send a single prompt**, get a response, and then send a completely **new prompt with no memory of the past**.

```
from google import genai

client = genai.Client(api_key="Paste your api key here")

response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents="How does AI work?"
)
print(response.text)
```

Observation:
This works, but the model has **no memory** of what you said earlier.
Every new message is treated as a standalone prompt — no continuity, no context.

But Now…

In real conversations, we don't restart every time.
We want the model to **remember what we said earlier**, just like a human would.

So now, we'll build a **multi-turn chat**, where the assistant uses previous messages to give **context-aware responses**.

```
from google import genai

# Initialize Gemini client
client = genai.Client(api_key="Paste your API key here")

# Create a new chat session using Gemini's chat API
chat = client.chats.create(model="gemini-2.5-flash")

# First message — sets initial context
response = chat.send_message("I have 2 dogs in my house.")
print(response.text)
```

```
# Second message — relies on memory (context)
response = chat.send_message("How many paws are in my house?")
print(response.text)

# Optional: Print the entire conversation history
print("\nConversation History:")
for message in chat.get_history():
    print(f'{message.role.capitalize()}: {message.parts[0].text}')
```

4.2 Step-by-Step Explanation:

1. from google import genai

You're importing the genai module — this is the **Gemini SDK** provided by Google, which allows developers to interact with Gemini models (like **gemini-2.5-flash**, etc.).

2. client = genai.Client(api_key="Paste your API key here")

You're creating an instance of the Gemini client using your **API key**.

- This API key is unique to you.
- It authenticates your requests to Google's Gemini models.

3. chat = client.chats.create(model="gemini-2.5-flash")

This line initializes a **new chat session** with the model gemini-2.5-flash.

Think of it like starting a new conversation. This chat object:

- Keeps a history of all messages
- Manages turns (user → model → user)
- Helps the model **retain context** across messages

4. chat.send_message("I have 2 dogs in my house.")

This is your **first message** — the user sets up some information.

- The SDK automatically treats this as coming from the **user**
- The model reads this and stores it in the chat history

5. print(response.text)

Prints the model's response to your message.
For example:

That's lovely! Dogs are great companions. What breeds are they?

6. chat.send_message("How many paws are in my house?")

This is the **second message** — and here's the magic:

- You're **not repeating** the "2 dogs" message.
- Yet the model can **recall it from context**, because it was stored in the same chat session.
- The model can now reason: *"Each dog has 4 paws, and there are 2 dogs — so 8 paws total."*

7. for message in chat.get_history():

This is an optional section that prints the **entire conversation history**, including:

- Role (user or model)
- Message content

It helps you **debug** or **visualize** what the model "remembers."

So far, we've seen that chat-based interactions don't use "memory" in the traditional sense.
The model doesn't **store** your previous messages like a human would.

Instead, we simulate memory through **prompt engineering** — by manually building a list of messages (**system**, **user**, **assistant**) and resending that structured prompt in every API call.

That's the secret behind multi-turn conversation:
The illusion of memory is actually **your prompt history**.

But here's the next critical question:

> "If we're building such long, structured prompts every time... how much can the model actually read and reason over?"
>
> "What happens if the prompt becomes too long, say, a 50-message conversation with large answers?"

This brings us to one of the **most important concepts** in working with LLMs:

i.e **Context Window**

5. Context Window:

Imagine you're having a conversation with someone through a narrow whiteboard window.

You write a message.
They write a reply.
Then you write the next one.
And so on...

But the board has a fixed length — say, it can only fit **3 screens of messages**.

As your conversation continues, the earliest messages get wiped off the board to make space for the new ones.

Now imagine you're asking them:

"Hey, remember that question I asked earlier?"

If it's still on the board — great, they can see it and respond properly.
If it's not? They'll say:

"Sorry, can you remind me?"

So, Just like that rolling whiteboard — where only the most recent messages are visible and the older ones get wiped off — LLMs work in a very similar way.

In the world of large language models, this limited whiteboard is called the **context window**

A **context window** is the **maximum number of tokens** a language model can "see" and process in a single request.

Let's say you're chatting with a model that supports 8K tokens.

If your structured message history crosses that:

- The oldest parts are **silently dropped**
- Assistant may forget key information
- Tone or behavior might change
- It may start asking questions it already answered earlier

The model isn't broken — it just **can't see what's no longer in the window.**

Everything you send —

- system message
- user inputs
- assistant replies
- formatting, whitespace, punctuation —

takes up space inside this limited window, Once you cross that limit?
The model **ignores the oldest parts** — they fall out of view.

5.1 Is the Context Window a Type of Memory?

No, the context window is not memory.
It's just a **temporary window of attention** — like a scroll the model reads once, then forgets.
If you want the model to 'remember' something, you have to **put it in the prompt again.**

> "If the model replies to my question and remembers what I said earlier… doesn't that mean it has memory?"

After all, it *feels* like the model remembers, right?

But here's the truth

> The model doesn't actually remember anything.
> It only "sees" what's currently visible in the context window — nothing more

5.2 Comparing the table of context window of different models:

Let's see the context window limit of different popular models:

Model	Max Context Window
GPT-3.5 Turbo	16,000 tokens
GPT-4 (standard)	8,192 tokens
GPT-4 (larger variant)	32,768 tokens
GPT-4 Turbo	128,000 tokens
GPT-4o (omni)	128,000 tokens

Gemini 1.5 Pro	128,000 tokens (1 M in private preview)
Gemini 2.0/2.5 Pro	1,000,000 tokens
Claude 2.1	200,000 tokens
Llama-4 Scout / Maverick	1,000,000+ tokens

Why It Matters

- A **larger context window** lets you send *more conversation or documents* in one go — useful for long chats, document summarization, codebase analysis, and more.
- **Smaller windows** need careful context management
- Learners should always **choose models based on their specific app requirements** — balancing context needs, cost, latency, and performance.

We've now understood that the **context window** is a temporary, invisible space — like a scroll — that the model uses to "read" the conversation before generating a response. It's not memory, but it plays a crucial role in helping the model stay relevant and coherent in its replies.

But not all context windows are created equal.

To make this more intuitive, let's divide models into two broad categories:

Large vs. Small Context Windows

- **Large context window**:
  These models can handle **very long conversations, documents, or chat histories** without much trouble. In most cases, you don't have to think too much about what to include or exclude — the window is big enough to hold it all.

- **Small context window**:
  Here's where things get tricky.
  The model can only "see" a limited slice of the conversation at a time.
  As your dialogue grows, the earlier parts of the conversation start falling off — which can cause

the assistant to:

- Forget what was said earlier
- Repeat itself
- Change tone or become inconsistent

So when working with smaller context windows, **we need to become strategic**.
We need to start thinking like conversation designers — deciding:

> "What information must the model always see?"
> "What parts can be safely dropped or compressed?"
> "How can we keep it coherent without exceeding the limit?"

That's where **Context Management Strategies** come in.

5.3 Context Management Strategies

Let's now dive deep into the most effective techniques to handle small context windows efficiently, without compromising on conversation quality

Strategy 1: Summarization — Compress the Past, Retain the Meaning

Instead of sending every old message back to the model, you distill the earlier conversation into a concise summary.

Let's say you're handing off a project to a new teammate.

You *don't* forward 60 Slack threads and every meeting transcript.
You say:

> "We've finished the UI, chosen React + Firebase, and launched version 1.2. Now we're working on analytics and crash reporting."

That's **summarization** — boiling down the *important parts* so the person joining can pick up *where you left off*.

LLMs need the same help. Instead of passing all prior chat turns, you can **summarize older conversations** and inject that summary into the prompt, either as a **system message** or appended to the user prompt.

This makes your application:

- More memory-efficient
- Better at retaining user goals
- Easier to scale for longer conversations

Benefits:

- Saves tokens
- Retains intent
- Helps model stay aligned with previous turns

Caution:
Summaries can drop nuance. Avoid compressing critical details too aggressively.

Strategy 2: Truncation — Keep It Fresh, Drop the Rest

Truncation is the practice of removing the oldest messages from a conversation to stay within the model's context window limit.

Rather than sending the entire conversation history to the model — which may exceed the maximum token capacity — you simply retain only the most recent N turns that are likely to be relevant right now.

Only keep the most recent N messages — everything else is dropped as you go.

For Example:

Think of a WhatsApp chat. You don't always scroll up to Day 1 — you're focused on the last few messages that are relevant now.

Similarly, your model only needs to see the latest 4–5 messages to continue the chat.

This creates a **sliding window** that always includes the latest exchanges, keeping the conversation contextually sharp and responsive.

Benefits:

- Very simple to implement
- Keeps conversation lightweight
- Prevents prompt overflow errors

Caution:
If you drop key information too early, the assistant may lose track of the goal or ask repeated questions.

Strategy 3: Prioritization — Keep What Matters Most

When dealing with a large context, not all information deserves equal attention.
Some elements are essential for task success, others are helpful but optional, and many are just noise.

Prioritization is the technique of:

- Identifying the most relevant or critical parts of a large prompt or context
- Including only those in your LLM call

- Dropping or compressing the rest

This helps you stay within token limits while preserving task clarity and response quality.

For Example:
Imagine you're an executive assistant preparing a briefing for the CEO before a board meeting.
You don't copy every email, every meeting note, every Slack message.
Instead, you include:

- Final decisions made
- Action items pending
- Financial figures or risks
- Important stakeholder feedback

You drop:

- Jokes
- Back-and-forth coordination
- Side remarks

That's Prioritization in action — you're curating high-value context for decision-making.

But now let's explore **how** to actually do it.

There are two primary ways:

1. Categorization (Manual or Rule-Based)

This method involves tagging or filtering your content based on predefined rules.

For example:

- You manually mark certain messages as "**priority**": "**high**" during message creation.

- Or, your app filters for messages containing key patterns like "**constraints**", "**requirements**", "**API key**" etc.

This method is simple and deterministic. You stay in control.

But What If Manual Rules Aren't Enough?

Manual categorization works well when your application is simple, the topics are limited, and the important parts are easy to define.

But in real-world use cases, that's rarely the case.

Imagine This:

You're building:

- a **customer support assistant** that handles finance, tech, and legal queries
- an **AI research agent** that reads thousands of papers and summarizes key findings
- a **project planning bot** that tracks updates from Jira, Slack, and email
- or even an **AI-powered writing assistant** that helps revise documents in specific tones

Now ask yourself:

> *How would you write static rules to detect what's relevant across such varied, complex, and changing content?*

So if writing hardcoded filters or tagging messages by hand isn't scalable...

> **How do we prioritize the most relevant information — automatically and intelligently?**

That's where the second strategy comes in:

2. Embedding-Based Relevance Scoring:

This method helps the model **figure out which parts of the conversation are most useful** — automatically.

You don't have to decide what to keep or what to skip.
Instead, you give everything to a tool (called an *embedding model*) that:

1. **Reads each message**
2. **Understands what it's about**
3. **Compares it with your current goal or question**
4. **Picks only the most helpful messages**

It's like asking a smart assistant:

> "Hey, here's all my notes — just pull out the ones that matter for this task."

The assistant doesn't just look for keywords — it understands the meaning behind each message.

Why it's useful:

- You don't have to write rules manually
- It works well even when the message doesn't use the exact same words
- It helps keep your prompt short without losing important context

So far, we've seen three powerful strategies — summarization, truncation, and prioritization — that help us manage the limited context window of a model."

But here's the interesting part: these strategies don't just help with *context limits*... they also help with something much more critical in real-world applications — **cost."**

As we know, every interaction with a language model — whether it's Gemini, GPT, Claude, or others — has a **cost**."

And this cost is **based on tokens** — that is, the number of tokens we send *to* the model, and the number it sends *back*."

So if your prompt is long — if it includes every chat history, instructions, previous assistant replies — you're **paying for all of that** every single time the model is called."

Now imagine this in a real app: one user has a 15-turn conversation with your assistant. If you keep sending all 15 messages back every time — you're racking up hundreds, maybe thousands of tokens — *per user.*"

And that's just one user.

That's why these context management strategies are not just **technical tricks** — they're your **first line of defense** against unnecessary API billing."

The same techniques we use to *manage context* also help us *control cost*."

5.4 Cost Management Strategies:

Let's look at them again — but this time, through the lens of saving money."

1. Summarization = Compress + Save

Let's say your user has had a 7-turn conversation about planning a Goa trip.

If you send all 7 turns every time, you're burning through 500–800 tokens in just the prompt. Now imagine multiplying that by hundreds of users daily.

With summarization, you reduce that 500-token backstory to maybe 50–80 tokens. You just retain the core idea.

From This:

User: I want to go to Goa for 3 days.

Assistant: Here's a suggested itinerary for North Goa...

User: What if we're going as a couple?

Assistant: In that case, you might enjoy romantic beach stays like...

User: Is Ashwem Beach good?

To This:

User: Planning a 3-day couple's trip to North Goa. Suggest romantic beaches and places to stay.

You've reduced 5 messages down to 1 — but the model still understands the same context. You pay less without losing context quality.

2. Truncation = Drop Irrelevant History

Imagine the conversation starts with small talk:
'Hi', 'How are you', 'Tell me about AI', and so on

Later, the user asks: 'Can you explain supervised learning?'

Do you need all the early greetings and general AI talk anymore? Probably not.

Instead of dragging the full message history into every new turn, just keep the latest 3–5 messages that are *still relevant.*"

3. Prioritization = Focus on High-Value Tokens Only

This is especially important when users mix serious questions with jokes, casual comments, or off-topic remarks.

Let's say you're building a health assistant. The user says:

User: I feel tired lately and get dizzy in the mornings.

Also, I stayed up late last night binge-watching a show 😅

Do you think I should get a check-up?

Now, you want the model to focus on the **health concern** — not the binge-watching emoji."

So you filter and forward only the important part:"

User: I feel tired lately and get dizzy in the mornings. Should I get a check-up?

> *That's prioritization — cutting the noise, keeping the signal.*

So now you've seen it — **context management isn't just about memory or model limits. It's your smartest tool for controlling cost.**"

Every time you reduce a long prompt to a sharp summary, drop an old message that no longer matters, or cut out irrelevant chatter — you're not just helping the model stay focused… **you're also saving money**."

And in real-world apps, that savings adds up — not just per user, but per session, per day, per month

So as you design conversations going forward, always ask yourself two things:"

1. *Does the model really need to see this?*
2. *Is it worth paying for?*

Because great prompt engineering isn't just about making the model smarter — **it's about making your application sustainable**

Important Note:

Imagine you've written your full application using OpenAI's API — everything from **chat.completions.create()** to tool/function calling.
Now suddenly, you want to try out the same logic with Google's Gemini model.

You may think:

"Do I have to rewrite everything from scratch?"
"Will Gemini even support this OpenAI-style format?"

Well, here's the good news:
Google's latest Gemini SDK is now compatible with OpenAI-style code, including function calling.

For Example:

Let's say you're testing or building on OpenAI models, but now you want to:

- See how Gemini responds to the same prompts
- Compare outputs from both LLMs
- Try Gemini's speed and cost benefits
- Avoid rewriting your codebase

Normally, you'd think:

"I'll have to send my code to another model and ask it to convert it."
But that's risky — unless you're 100% sure your original code was perfect.

Instead, Gemini offers native support for OpenAI-style function calls. That means you can directly reuse your OpenAI code with almost zero changes.

That's it. No code migration. No conversion.
Just plug and play.

What You Need to Change for Gemini:

1. Set your API key to Gemini's key

2. Point the library to Gemini's endpoint:

```
client = OpenAI(
    api_key="YOUR_GEMINI_API_KEY",
    base_url="https://generativelanguage.googleapis.com/v1beta/openai/"
)
```

3. Gemini model name

```
response = client.chat.completions.create(
    model="gemini-2.5-flash",   # or gemini-2.0-flash etc.
    messages=[…],
    tools=[…],                  # function definitions
    tool_choice="auto"          # Gemini supports tool calling
)
```

The rest of your code—whether it's function declarations, calling logic, or tool handling—stays exactly the same as it would with GPT.

Conclusion

1. **Limitations of Prompt-Based Generation**
   Traditional prompt-response models lack memory, role-awareness, and behavioral consistency—making them unsuitable for scalable, multi-turn applications like chatbots and virtual assistants.

2. **Power of Chat Completion APIs**
   Chat Completion introduces structure by using role-based messages (**system**, **user**, **assistant**) to simulate human-like conversations with memory, tone, and context awareness—all within a single API call.

3. **System Role Enables Persistent Behavior**
   The **system** role is a foundational control tool that sets the assistant's persona, tone, and constraints at the beginning of the session—eliminating the need to repeat instructions and improving response consistency.

4. **No Real Memory—Only Prompt Engineering**
   LLMs do not store any conversation history. The illusion of memory is achieved by continuously resending the structured message history within the model's context window for every interaction.

5. **Context Management is Crucial for Small Windows**
   For models with limited context capacity, techniques like **summarization**, **truncation**, and **prioritization** (manual or embedding-based) help maintain relevance and coherence by managing which messages the model "sees"